

Codes Sources Structurés – Projet G-Lab

Réalisé par : Abdallah ATGUIRI

Date : Juillet 2026

Arborescence du projet

...

Projet final G-Lab/

```
|
|— pom.xml # Configuration Maven (dépendances, build,
JAR)
|— run.sh # Script de compilation et lancement rapide
|— README.md # Documentation technique du projet
|
|— lib/ # Bibliothèques JDBC (téléchargées
automatiquement)
|   |— sqlite-jdbc-3.45.1.0.jar
|   |— slf4j-api-2.0.12.jar
|   |— slf4j-simple-2.0.12.jar
|
|— src/main/java/com/glab/ # ← CODE SOURCE PRINCIPAL
|   |— GLabApp.java # Point d'entrée de l'application
|   |
|   |— model/ # COUCHE MODÈLE (POO)
|       |— Ressource.java # Classe abstraite – socle commun
|       |— EquipementAutomate.java # Sous-classe : automates industriels
|       |— EquipementDrone.java # Sous-classe : drones embarqués
|       |— DocumentationTechnique.java # Sous-classe : documentation
|   |
|   |— controller/ # COUCHE CONTRÔLEUR (logique métier)
|       |— RessourceController.java # Validation, ajout, suppression,
filtrage
|
|   |— database/ # COUCHE PERSISTANCE (JDBC)
|       |— DatabaseManager.java # Connexion SQLite, création de table
|       |— RessourceDAO.java # CRUD polymorphe (INSERT, SELECT,
DELETE)
|
|   |— view/ # COUCHE VUE (interface graphique Swing)
|       |— FenetrePrincipale.java # Fenêtre principale, tableau, formulaire
|
|— target/ # Fichiers compilés (générés par Maven)
|   |— g-lab-1.0.0.jar # JAR exécutable autonome
|
|— livrables/ # Documents de rendu du projet
|   |— Rapport_Conception_00.md
|   |— Structure_Code_Source.md
|   |— Presentation_G-Lab.pptx
|   |— README.md
|
|...
```

Description fichier par fichier

Point d'entrée

Fichier	Package	Lignes	Description
GLabApp.java	com.glab	~28	Initialise la base SQLite, configure le Look & Feel système, lance la fenêtre Swing via `EventQueue.invokeLater`.

Couche Modèle – `com.glab.model`

Fichier	Classe	Type	Responsabilité
----- ----- -----			
`Ressource.java`	`Ressource`	**abstract**	Attributs communs (id, désignation, emplacement, quantité). Méthodes abstraites `getType()` et `getDiagnostic()`.
`EquipementAutomate.java`	`EquipementAutomate`	extends Ressource	Marque, nombre d'entrées/sorties, protocole réseau (Modbus, Profinet...).
`EquipementDrone.java`	`EquipementDrone`	extends Ressource	Masse (kg), autonomie de vol (min), type de capteur embarqué.
`DocumentationTechnique.java`	`DocumentationTechnique`	extends Ressource	Auteur/constructeur, nombre de pages, lien vers le manuel PDF.

Couche Contrôleur – `com.glab.controller`

Fichier	Classe	Responsabilité
----- ----- -----		
`RessourceController.java`	`RessourceController`	Orchestration entre vue et DAO. Validation des champs (`ValidationException`). Méthodes `ajouterAutomate/Drone/Documentation`, `supprimer`, `filtrer`, `chargerRessources`.

Couche Base de données – `com.glab.database`

Fichier	Classe	Responsabilité
----- ----- -----		
`DatabaseManager.java`	`DatabaseManager`	Singleton utilitaire. URL JDBC `jdbc:sqlite:glab_parc.db`. Création automatique de la table `ressources`.
`RessourceDAO.java`	`RessourceDAO`	`ajouter()`, `mettreAJour()`, `supprimer()`, `chargerToutesLesRessources()`. Sérialisation/désérialisation polymorphe avec `instanceof` et `switch`.

Couche Vue – `com.glab.view`

Fichier	Classe	Responsabilité
----- ----- -----		
`FenetrePrincipale.java`	`FenetrePrincipale`	`JFrame` principale. Panneau recherche (filtrage temps réel), tableau `JTable`, formulaire adaptatif `GridLayout`, boutons Ajouter/Supprimer/Rafraîchir.

Conventions de codage respectées

Convention	Application
----- -----	
Nommage français	Classes et méthodes en français métier (`FenetrePrincipale`, `chargerRessources`)
Packages par couche	`model`, `view`, `controller`, `database`
Encapsulation	Tous les attributs sont `private` avec accesseurs
Javadoc	Chaque classe documentée avec un commentaire de description
Try-with-resources	Toutes les ressources JDBC fermées automatiquement
PreparedStatement	Protection contre les injections SQL
Séparation MVC	La vue n'accède jamais directement à la DAO

Commandes de compilation et exécution

```
```bash
Méthode 1 – Maven (recommandée)
mvn compile # Compiler
mvn exec:java # Lancer l'application
```

```
mvn package # Créer le JAR exécutable
```

```
Méthode 2 – Script shell
bash run.sh
```

```
Méthode 3 – JAR autonome
java -jar target/g-lab-1.0.0.jar
```
```

```
---
```

```
## Dépendances externes
```

| Bibliothèque | Version | Usage |
|--------------|----------|--|
| ----- | ----- | ----- |
| sqlite-jdbc | 3.45.1.0 | Pilote JDBC pour SQLite |
| slf4j-api | 2.0.12 | API de journalisation (requis par sqlite-jdbc) |
| slf4j-simple | 2.0.12 | Implémentation simple de SLF4J |

Les bibliothèques Swing, JDBC et AWT sont fournies par le JDK (aucune dépendance externe pour l'IHM).